Università di Brescia

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Corso di Laurea in Ingegneria Informatica

Relazione del progetto di Robotica

Maze Exploration Search Analysis

Anno di Corso 2025-2026

Docente del corso:

Prof. Enrico Scala

Prof. Luigi Gargioni

Studenti:

Francesco Guarda

Andrea Moro

Contents

1 Introduction

This project treats goal-directed maze exploration as an **online replanning problem**, in the classical Micromouse paradigm: the agent knows the start and goal cells in advance but not the maze layout, and must interleave sensing, (re)planning, and acting to reach the goal while minimizing exploration effort. It adopts the **freespace assumption** — every unexplored cell is optimistically treated as passable until a wall is sensed — with a *replanning event* triggered whenever new information invalidates the current plan. The central question is which replanning strategy handles this sense–plan–act loop most effectively.

Two strategies are evaluated head-to-head — A* (full replanning) and D*-Lite (incremental replanning) — isolating the effect of *how* a plan is repaired when a new wall is discovered. Both run against a common exploration framework built for this comparison: a shared interface lets either algorithm drive the same sense-plan-act loop, and a headless extension of the MMS simulator makes fully automated, GUI-free evaluation at scale practical. Multi-goal exploration is a first-class capability of this framework, not a special case of single-goal search, and exploration difficulty is manufactured deliberately — through where goals are placed, not through choice of maze — via a **detour index** extended here to nested multi-goal scenarios. A 440-run controlled comparison then evaluates D*-Lite as an *incremental* search algorithm that reuses prior search effort across replanning events, not merely as an alternative heuristic to A*.

2 Background

The Micromouse problem and the MMS simulator. The Micromouse competition tasks a small autonomous robot with exploring an unknown grid maze from a fixed start to a fixed goal region. This project uses `mms` (mackorone, 2024), which reproduces that setting: a GUI for visualization, driving the algorithm through a text-based `stdin`/`stdout` protocol of wall queries, moves, turns, and display commands. It was adopted for its standard, competition-faithful interface, built-in visualization, and existing Python bindings, extended in this project (§3).

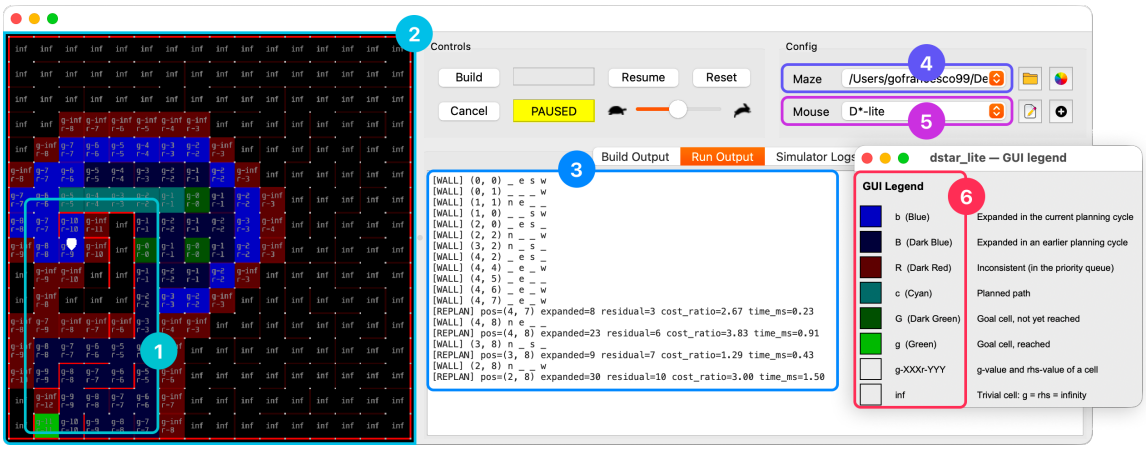

Figure 1: Annotated `mms` GUI during a running D*-Lite session: (1) wall segments and per-cell overlay text rendered by the algorithm; (2) the maze grid; (3) the Run Output panel, logging sensing and replanning events; (4) the maze-selection path; (5) the algorithm-selection field; (6) the algorithm's GUI legend window.

Search under partial observability. Under the freespace assumption, an agent plans as if every unsensed cell were open, then repairs its plan when a sensed wall proves otherwise — the optimistic-planning principle behind the D* family of incremental replanners (Stentz, 1994). This report compares two heuristic search strategies built on that principle: A* (Hart et al., 1968), the classical best-first

baseline, and D*-Lite (Koenig & Likhachev, 2002, 2005), designed explicitly to reuse previous search results rather than resolve the whole problem from scratch — both described mechanistically in §3.3.

3 Methodology

3.1 A common interface for exploration algorithms

`BaseAPI` is an abstract contract — wall sensing, movement, display, reset — decoupling an exploration algorithm from its I/O backend. `BaseAlgorithm` implements the shared sense → plan → act → log loop on top of it, together with goal resolution (an explicit list, random goals, or a default centre area) and the metrics hooks common to every concrete algorithm. Adding a third algorithm to this project therefore means implementing one class against this interface, with no change to execution, logging, or evaluation tooling.

3.2 Headless execution for automated evaluation

A second backend, `SimAPI`, implements the same `BaseAPI` contract entirely in memory, without the MMS GUI process. This is what makes a large deterministic batch campaign practical: the same algorithm code runs unmodified under either backend, and was cross-checked against the real MMS GUI to confirm behavioural equivalence before the campaign in §4 was run headlessly at scale.

3.3 Exploration algorithms

A* treats every replanning event as an independent search: after each newly discovered wall invalidates the current plan, it rebuilds an open list and closed list from scratch, rooted at the robot's current position, and expands nodes by increasing f = g + h until a goal is popped; nothing survives into the next event. Movement cost is 1 per traversable edge; multi-goal exploration greedily re-targets the nearest remaining goal once one is reached, discarding all information about the other goals as it restarts.

D*-Lite instead keeps two values alive per cell across the whole episode: `g(s)`, the current best-known cost from `s` to the goal, and `rhs(s)`, a one-step lookahead equal to the cheapest neighbour's `g` plus one. A cell is *consistent* once `g(s) = rhs(s)`; the algorithm keeps every reachable cell consistent except where a change has just occurred. When a wall is newly confirmed, only the cells touching that edge have their `rhs` recomputed and are pushed onto a priority queue as inconsistent; repair then propagates outward from those cells until consistency is restored, touching only the region the wall affects. Everywhere else `g` and `rhs` are untouched and reused as-is in the next planning cycle. A reached goal is retired — its `rhs` set to infinity — and the search re-targets the next one without discarding this retained state.

The experimental campaign (§4) fixes **both** algorithms to the Manhattan heuristic: admissible and consistent, O(1) per query rather than a fresh graph search, and matching D*-Lite's own incremental `km`-based heuristic update — so neither algorithm's node or time counts are confounded by heuristic-recomputation overhead, and the comparison stays isolated to *how* each repairs its plan.

3.4 Multi-goal exploration

Goals may be an explicit list, a random sample, or an automatically placed set (§3.5); both algorithms visit them in greedy nearest-goal order, though the search mechanics producing that order differ. A*'s search is rooted at the current position and terminates at the first goal popped from its open list, discarding all partial information about the other goals once that episode ends. D*-Lite's search is instead rooted at the *entire* remaining goal set simultaneously (every unreached goal initialised

with `rhs = 0`), so once the nearest goal is reached, the `g`/`rhs` state already built toward the others remains valid and is reused directly — an amplification, specific to multi-goal exploration, of the reuse advantage above. With no goals specified, both algorithms fall back to the standard Micromouse 4-cell centre goal area (first arrival ends the run); this default is not used by the experimental campaign, where every scenario is placed automatically (§3.5).

3.5 Modeling exploration difficulty: the detour index

For a reference cell and a candidate cell, the **detour index** is the ratio of true in-maze (BFS) distance to straight-line (Manhattan) distance — a cell that looks close but is actually far behind walls scores highest, exactly the configuration that defeats a greedy planner working from a partial map (the *route factor/circuity* of spatial-network analysis: Barthélemy, 2011; Gastner & Newman, 2006). The first goal maximises detour from the start; each additional goal maximises the *minimum* detour over the start and every previously placed goal, so a k-goal scenario stays jointly deceptive rather than just individually so; scenarios are **nested** — a k-goal scenario's first j goals equal the j-goal scenario's goals. The metric is used here to manufacture exploration effort through goal placement, not to grade maze difficulty (§4.1): the same 55-maze corpus is reused at every k, and it is where goals are placed within it, not a choice of "harder" mazes, that scales the work asked of both algorithms. Its normalisation is known to favour goals close to the start (§5).

4 Experimental Evaluation

4.1 Setup

The corpus is 55 standard Micromouse competition mazes (Weisberg), all 16×16 (256 cells), filtered to guarantee full connectivity from the start so every goal is reachable. Four goal-count scenarios are placed automatically per maze, k = 1..4, by detour-index placement (§3.5). The full factorial, headless batch campaign — 2 algorithms × 55 mazes × 4 scenarios — produced **440 runs**, each logging run-level scalars and a per-event replanning record (nodes expanded, planning time, residual distance to goal, memory occupancy).

Every metric except wall-clock planning time is exactly reproducible run to run: the campaign is a full census of the (algorithm × maze × scenario) space rather than a sample, so the analysis below is descriptive and paired rather than inferential — no p-values or resampling are reported. Planning time is the only quantity with genuine measurement noise. Both algorithms are complete searches under the freespace assumption, so reaching every goal on a fully connected maze is guaranteed rather than a finding; all 440 runs did so.

4.2 Goal-placement difficulty across the maze corpus

Detour-index placement is designed to progressively intensify exploration effort as k grows, not to manufacture increasingly difficult individual goals: because placement is nested, every k-goal scenario shares the same first goal, and each subsequent goal stays comparably deceptive relative to what is already known rather than becoming harder in an absolute sense (§3.5). Aggregate exploration effort therefore grows with k across the whole corpus (§4.3–4.5), even though no single goal is engineered to be harder than the last. Table 1 spans the resulting range of first-goal placements across the corpus, from genuinely remote, high-scoring mazes to a case where a high score reflects only the goal's proximity to the start rather than real difficulty — a pattern that recurs in 26 of the 55 mazes (§5). Figure 2 shows the score map goal placement maximises at each step, k = 1..4, for maze 2009japan: high-scoring cells sit immediately behind walls close to the reference set.

Maze	Manhattan dist.	Real (BFS) dist.	Detour score
zigzag	3	107	35.7
2015japan	3	75	25.0
2017apec	5	99	19.8
museum	1	5	5.0

Table 1: First-goal (k=1) placement for four mazes spanning the score range: zigzag, 2015japan and 2017apec are genuinely remote as well as high-scoring; museum scores comparably high only because its Manhattan distance is 1. Every nested k $\geq$ 1 scenario shares this same first goal.

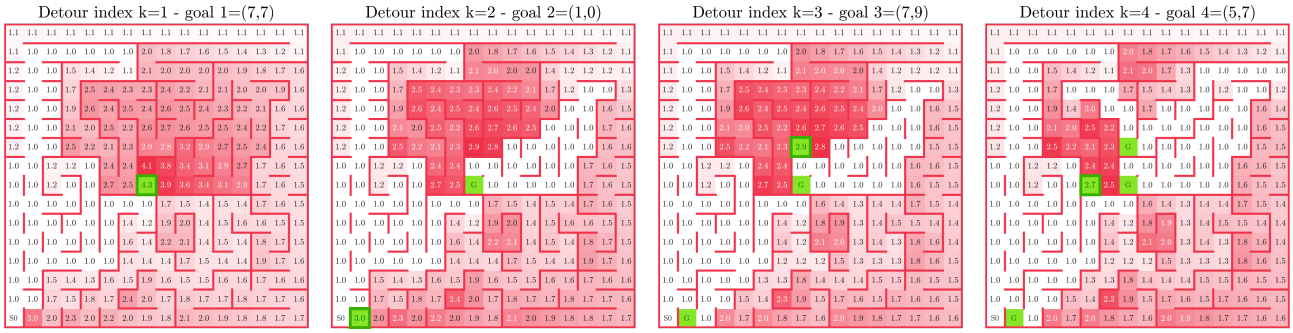

Figure 2: Score map maximised by goal placement at each step k = 1..4, for maze 2009japan.

4.3 Replanning cost: nodes expanded and wall-clock planning time

D*-Lite expands **2.3–3.0× fewer nodes** than A* at every k and accumulates **2.6–3.1× less** cumulative planning time (Figure 3) — a consistent win on both measures, not a trade-off: fewer nodes in 212 of the 220 matched (maze, k) pairs, lower time in 219 of 220. This is the direct, expected consequence of the incremental-repair mechanism in §3.3: only the cells touched by a newly discovered wall are reprocessed, so cost tracks the size of the change rather than the size of the whole remaining problem. Nodes expanded is the algorithm-intrinsic, implementation-independent measure of search effort; planning time is useful corroboration but remains hardware/implementation-dependent in principle — here the two agree on every ordering.

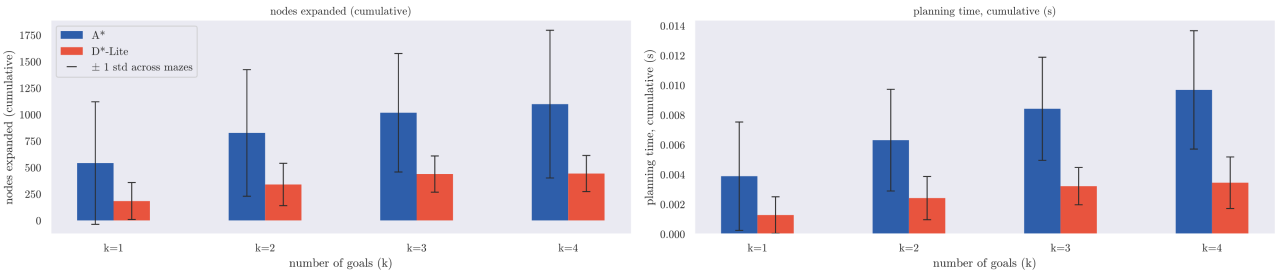

Figure 3: Cumulative nodes expanded and cumulative planning time, A* vs. D*-Lite, grouped by goal-count scenario (mean across the 55 mazes, ±1 std-dev error bars).

4.4 Search-cost scaling: evidence for incremental reuse

Regressing nodes expanded on residual distance to goal per replanning event (dense band, residual distance $\leq$ 30, 99.3% of the 18,176 logged events): A*'s slope is steeper than D*-Lite's at every k and the gap widens with k — 1.83 vs. 0.66 at k=1, up to 2.75 vs. 0.81 at k=4; pooled, A*'s slope (2.32) is 3.1× D*-Lite's (0.76) (Figure 4). This is direct empirical support for D*-Lite's central claim (§3.3): repair cost scales with the *size of the region a change affects*, not with the size of the whole remaining problem, whereas A*'s from-scratch cost grows with the full remaining search space. Table 2 shows

the same divergence bin by bin: the ratio climbs from 1.4× nearest the goal to 2.6× by 15–20 cells away, then holds rather than widening further — D*-Lite's relative advantage is therefore bounded and predictable, not unbounded with distance.

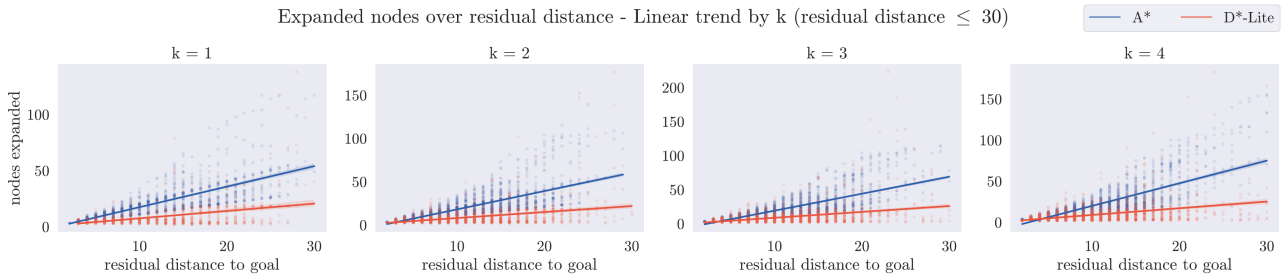

Figure 4: Nodes expanded vs. residual distance to goal, per replanning event, faceted by goal-count scenario (k=1..4); linear trend with 95% confidence band.

Residual distance	μ_{A*}	$\text{ci}^{95\%}_{A*}$	$\mu_{D*\text{-Lite}}$	$\text{ci}^{95\%}_{D*\text{-Lite}}$	Ratio
(0, 5]	6.6	[6.5, 6.7]	4.7	[4.6, 4.8]	1.4×
(5, 10]	13.5	[13.3, 13.6]	6.9	[6.7, 7.0]	2.0×
(10, 15]	22.8	[22.4, 23.1]	9.1	[8.7, 9.4]	2.5×
(15, 20]	34.8	[33.7, 35.9]	13.2	[12.4, 14.0]	2.6×
(20, 25]	52.6	[49.9, 55.3]	20.0	[17.1, 22.9]	2.6×
(25, 30]	77.7	[70.2, 85.3]	30.7	[23.9, 37.4]	2.5×

Table 2: Mean nodes expanded with 95% confidence interval by residual-distance bin, restricted to the dense band used for the fit above (99.3% of all logged events); three sparser bins beyond 30 cells (13–42 events each) are excluded as too noisy to support a reliable estimate.

4.5 Memory footprint of search structures

The two algorithms trade cost for statelessness in opposite directions (Figure 5). On a single run of maze 00japan at k=4 — the run whose D*-Lite peak sits closest to the corpus median at that k — A*'s open+closed set oscillates in a low band and ends close to where it started (peak 122, final 15, of 256 cells): it resets every replanning event. D*-Lite's working set — cells currently holding a finite `g` or `rhs` — instead climbs toward a plateau near 80% of the maze and stays there (peak 202, final 180 on the same run); the climb is not perfectly monotonic, since a newly discovered wall resets some cells' `g` to infinity until a new shortest path re-supports them, but the trajectory is unmistakably one of accumulation rather than reset. The pooled trend (Figure 5, right) confirms this holds across the corpus, not just the run shown: D*-Lite's occupancy rises steadily with event count while A*'s stays flat.

Table 3 summarises this across all 220 runs per algorithm: D*-Lite's median peak occupancy is **2.6× A*'s** (182.5 vs. 69.0 cells) and its median per-run mean is **4.0× A*'s** (116.58 vs. 29.51); no A* run's peak exceeds 75% of the maze, while 31 of 220 D*-Lite runs exceed 90% and four fill it completely. The penalty is mildest at k=1 (1.7×, since short runs end before the working set saturates) and settles at 2.4–2.9× from k=2 on. This reverses the ordering of §4.3: A* buys its bounded footprint at the price of repeated search, D*-Lite buys cheap repair at the price of retained state — neither algorithm dominates outright.

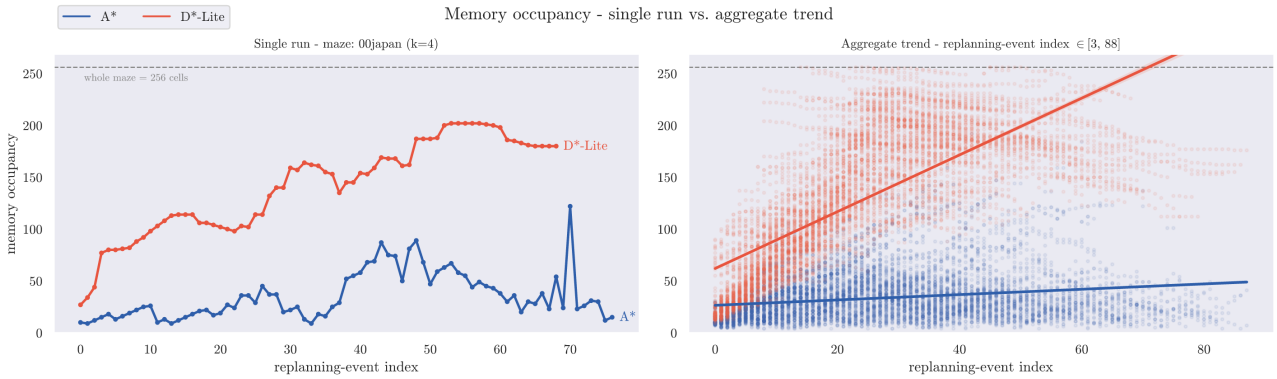

Figure 5: Memory occupancy of search structures over successive replanning events. Left: a single run (maze 00japan, k=4) — bounded and resetting for A*, growing toward a plateau for D*-Lite. Right: the same trend pooled across all 220 runs per algorithm in the corpus.

Algorithm	Peak — median	Peak — σ	Mean — median	Mean — σ
A*	69.0	38.76	29.51	10.35
D*-Lite	182.5	71.63	116.58	47.21

Table 3: Peak and per-run mean memory occupancy (cells, out of 256), median and standard deviation across all 220 runs per algorithm.

5 Conclusions and Future Work

On the metrics that matter, D*-Lite's incremental repair consistently outperforms A*'s from-scratch replanning: fewer nodes expanded per event and less cumulative planning time overall, with the gap widening as replanning distance grows (§4.3–4.4). That advantage is paid for in memory: D*-Lite's working set grows toward roughly 70% of the maze and stays there, against a bounded, resetting footprint for A* (§4.5). Neither algorithm dominates outright — the right choice depends on which resource is scarcer, computation or memory, which on micromouse-class embedded hardware is not a foregone conclusion.

The main limitation of this evaluation is the detour index's own normalisation, which biases goal placement toward the start rather than calibrating absolute difficulty (§4.2; full discussion in the repository); a start-proximity correction to the metric would remove this bias and is the most direct next step, alongside extending the shared exploration interface to additional algorithms — weighted or anytime variants, in particular — now that adding one requires implementing only a single class against it (§3.1). The full implementation, maze corpus, and experimental logs are available in the project repository (Guarda & Moro, 2026).

References

- **MMS simulator:** mackorone, *mms — A Micromouse Simulator*, v1.2.0, MIT License, GitHub (2024).
- **Maze corpus:** J. Weisberg, *Micromouse Maze Collection*, tcp4me.com.
- **A*:** P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107, 1968.
- **D* / the freespace assumption:** A. Stentz, "Optimal and Efficient Path Planning for Partially-Known Environments," *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 1994.
- **D*-Lite:** S. Koenig and M. Likhachev, "D* Lite," *Proc. AAAI/IAAI*, 476–483, 2002; and S. Koenig and M. Likhachev, "Fast Replanning for Navigation in Unknown Terrain," *IEEE Transactions on Robotics*, 21(3), 354–363, 2005.
- **Detour index / route factor:** M. Barthélemy, "Spatial Networks," *Physics Reports*, 499(1–3), 1–101, 2011; M. T. Gastner and M. E. J. Newman, "The Spatial Structure of Networks," *European Physical Journal B*, 49(2), 247–252, 2006.
- **Rob-26-MazeSolver repository:** F. Guarda and A. Moro, *Robotica 2026 - Maze Solver Project*, software, MIT License, GitHub (released 2026-07-29).

Acknowledgements. The authors acknowledge the use of large language model (LLM)-based artificial intelligence tools during the preparation of this report. These tools were used exclusively to improve the clarity and readability of the manuscript and to assist with source code development and debugging. All technical content, implementation decisions, experimental methodology, and conclusions remain the sole responsibility of the authors.